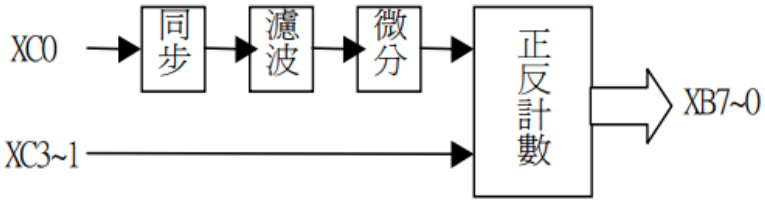
FPGA 數位邏輯系統設計 作業七

第七章計數處理

通訊 4A 0106C033 桑瑋佑

一 實驗目的:

結合同步、濾波和微分電路，建立案件控制的同步計數器。



二 步驟:

實驗時，先用 LoadPGA 程式將 ch07.rbt 的電路檔載入 FPGA 中，試著依定義來控制按鍵，再由 LED 的顯示狀況去確認 FPGA 是否正確執行。

其中四個 按鍵訊號 分別控制 ：

XC(0)：計數輸入，每次按鍵控制計數器加一或減一。

XC(1)：正反控制，當按下時選擇下數，放開則選擇上數。

XC(2)：計數禁止，當按下時不允許 XC(0)輸入訊號。

XC(3)：計數歸零，當按下時計數值歸零。

三 結果:以下為程式碼，為 Ch07.vhd 檔

```
--The IEEE standard 1164 package, declares std_logic, rising_edge(),
etc.
```

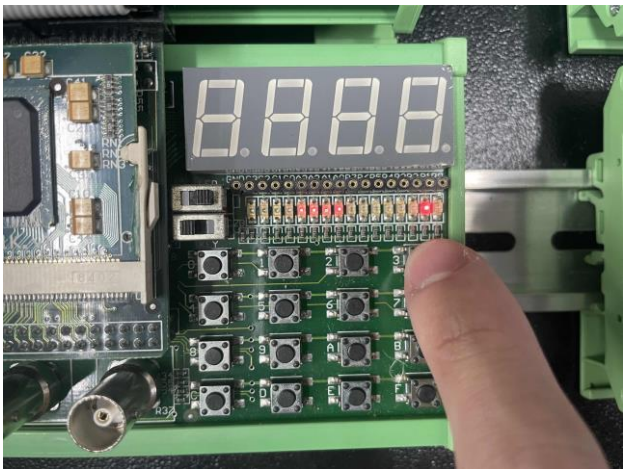
```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

--
--*******************************************************
-- CH07 : practise synchronize UP/DOWN counter
--                    designed by Pei-Chong Tang, Jan. 1999
--*******************************************************
--
entity CH07 is
     port
     (    CLK : in  STD_LOGIC;
          XB  : out STD_LOGIC_VECTOR (7 downto 0);
          XC  : in  STD_LOGIC_VECTOR (3 downto 0)
     );
end CH07;

architecture CH07_ARCH of CH07 is
     signal SAMPLE,FLT,DIF : STD_LOGIC;
begin
--
--*******************************************************
--
-- 17-BIT free counter
--
FREE_COUNTER : block
     signal Q   : STD_LOGIC_VECTOR (16 downto 0);
     signal DLY : STD_LOGIC;
begin
     process (CLK)
     begin
          if CLK'event and CLK='1' then    --CLK rising
               DLY <= Q(16);               --delay
               Q   <= Q+1;                 --UP counter
          end if;
     end process;
```

```
     SAMPLE <= Q(16) and not DLY;          --differential
end block FREE_COUNTER;
--
--*****************************************************
--
-- keyboard debouncing
--
DEBOUNCING : block
     signal D0,D1 : STD_LOGIC;
begin
     process (CLK)
     begin
          if CLK'event and CLK='1' then    --CLK rising
               if SAMPLE='1' then          --sampling control
                    D1<=D0; D0<=not XC(0); --delay
                    FLT <= ((D0 and D1) or FLT) and (D0 or D1); --RS-
F/F
               end if;
          end if;
     end process;
end block DEBOUNCING;
--
--*****************************************************
--
-- differential signal
--
DIFFERENTIAL : block
     signal D0,D1 : STD_LOGIC;
begin
     process (CLK)
     begin
          if CLK'event and CLK='1' then    --CLK rising
               D1<=D0; D0<=FLT;            --delay
          end if;
     end process;
     DIF <= D0 and not D1;                 --differential
end block DIFFERENTIAL;
--
```

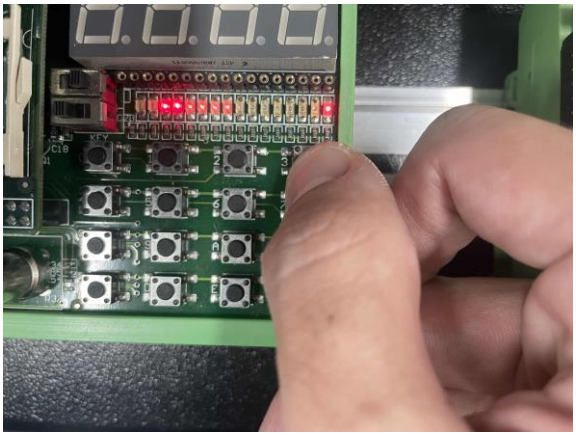
```
--*****************************************************
--
-- 8-BIT synchronize up/down counter
--
UPDOWN_COUNTER : block
     signal Q : STD_LOGIC_VECTOR (7 downto 0);
     signal RST,EC,DIR : STD_LOGIC;
begin
     process (CLK,RST)
     begin
          if RST='1' then                  --counter reset
               Q <= "00000000";            --Q=0
          elsif CLK'event and CLK='1' then --CLK rising
               if EC='1' then              --CLK enable
                    if DIR='0' then        --UP/DOWN direction
                         Q <= Q+1;         --UP   counter
                    else
                         Q <= Q-1;         --DOWN counter
                    end if;
               end if;
          end if;
     end process;
     RST <= not XC(3);                     --RST signal
     EC  <= DIF and XC(2);                 --EC  signal
     DIR <= not XC(1);                     --DIR signal
     XB  <= not Q;                         --output signal
end block UPDOWN_COUNTER;
--
--*****************************************************
-- end of architechture
--
end CH07_ARCH;
```

以下為成果圖:

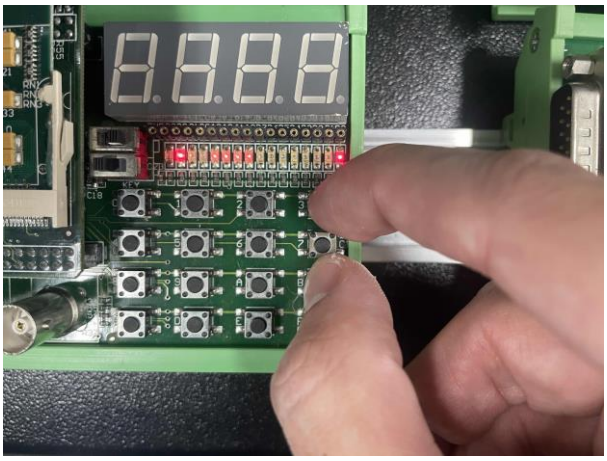
利用 4 組按鍵輸入 XC(3~0)來控制計數器，而利用 8 組 LED 輸出 XB(7~0)顯示計數結果。



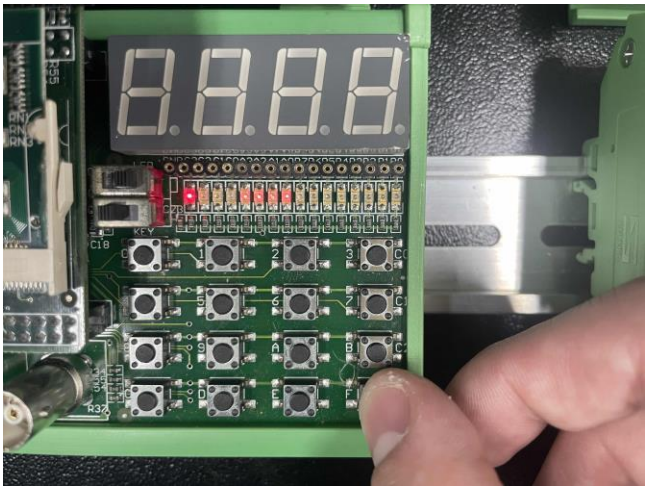
此為按下 XC(0)，計數器加一。XB1 亮與 XB(0)不亮變成 XB(0)和 XB(1)都亮



此為同時按下 XC(1)和 XC(0)，計數器做下數，計數器減一。



此為同時按下 XC(2)和 XC(0)或同時按下 XC(2)和 XC(1)，按下 XC(2)時不允許訊號輸入，因此不論上數或下數均不會改變輸出結果。



最後為按下 XC(3)，計數器值歸零。

四 程式重點:

1. FREE_COUNTER : block --自由計數器宣告

```
        signal Q : STD_LOGIC_VECTOR (14 downto 0);  --15 位元計數器
        signal DLY : STD_LOGIC;                     --內部延遲訊號
    begin
        process (CLK)                               --順序邏輯開始
        begin
            if CLK'event and CLK='1' then           --上緣觸發
                DLY <= Q(14);                       --延遲電路
                Q <= Q + 1;                         --正向計數器
            end if;
        end process;                                --順序邏輯結束
        SAMPLE <= Q(14) and not DLY;                --微分電路
     end block FREE_COUNTER;                        --宣告結束
```

此為 15 位元的自由計數器，產生一組 125Hz 的抽樣訊號稱為 SAMPLE 訊號。並利用微分電路將脈衝縮小到一個 CLK 週期寬，提供給案件彈跳消除電路使用。

2. DEBOUNCING : block --彈跳消除電路宣告

```
        signal D0,D1 : STD_LOGIC;                   --內部延遲訊號
    begin
        process (CLK)                                   --順序邏輯開
        始
        begin
            if CLK'event and CLK='1' then           --上緣觸發
                if SAMPLE='1' then                  --抽樣控制
                    D1<=D0; D0<=not XC(0);          --延遲電路
```

```
                                                    --RS 正反器
                    FLT <= ((D0 and D1) or FLT) and (D0 or D1);
                end if;
            end if;
        end process;                                --順序邏輯結
        束
    end block DEBOUNCING;                           --宣告結束
```

其次利用 8ms 周期的 PRD 抽樣訊號，執行 XC(0)按鍵訊號的彈跳消除電路。

彈跳消除電路除了抽樣處理之外，就是一個 RS 正反器:

Q <= (S or Q) and not R;　連續兩次 1 設定 S，連續兩次 0 復歸 R

Q <= ((D0 and D1) or Q) and not (not D0 and not D1);

由於(not (not X and not Y)) 等於 (X or Y)

Q<=((D0 and D1) or Q) and (D0 or D1);

```
3. DIFFERENTIAL : block                             --微分電路宣
   告
        signal D0,D1 : STD_LOGIC;                   --內部延遲訊
        號
   begin
        process (CLK)                               --順序邏輯開
        始
        begin
            if CLK'event and CLK='1' then           --上緣觸發
                D1<=D0; D0<=FLT;                    --延遲電路
            end if;
        end process;                                --順序邏輯結
        束
        DIF <= D0 and not D1;                       --微分電路
   end block DIFFERENTIAL;                          --宣告結束
```

接下來是微分電路，將 FLT 訊號的寬度縮小到一個 CLK 時鐘訊號的週期寬度。

五 重點提醒

1. 在並行模式下可以使用‘when-else‘指令，但在‘process‘中因為是順序執行模式，此時若使用‘when-else‘指令則會出現錯誤。

2. 延遲電路屬於順序邏輯，因此需要放在‘process‘指令內；而

微分計算屬於組合邏輯，必須放在‘process‘ 指令外。

3. 由於同步電路的延遲效應，計數器的 CNT 輸出值比 DIF 微分信號晚了一個 CLK 週期。

六 心得:

這次課程的難度顯著提升，開始加入了同步、濾波和微分等不同功能的電路，各自完成不同的工作。學習過程中，我們掌握了越來越多的設計方法，並能創建更多元的指令。這次課程先教我們計數器，然後逐一分析我們可能面臨的問題，以及對應的解決方法。透過這樣的步驟，我們不僅學到新知識，還學會解決實際遇到的挑戰。課程強調理論和實踐並行，學習效果很好；不過，由於知識量較大，我需要多加複習，避免遺忘。